UCD Michael Smurfit Graduate Business School

FIN 42110: DATA SCIENCE FOR TRADING & RISK MANAGEMENT

High-Frequency Bitcoin Volatility Estimation and Forecasting: A Realised Variance Approach



Professor Richard Mc Gee

Shree, Kiara - 25222331

TABLE OF CONTENTS

PART 1 - DATA LOADING, CLEANING AND PLOTTING **3**
1.1 Data Description 3
1.2 Price by Exchange 3
1.3 Data Anomalies 3
1.4 Daily Volume by Exchange 4
PART 2: 5-MINUTE VWAP & REALIZED VARIANCE **4**
2.1 Five-Minute Price Construction 4
2.2 Missing Interval Analysis 4
2.3 Five-Minute Log Returns 5
2.4 Hourly Realised Variance 5
2.5 Daily Realised Volatility 5
PART 3 - FORECASTING **6**
3.1 Forecasting Hourly Realised Variance Using Lagged RV 6
3.2 Garman-Klass Volatility as Predictor 6
3.3 Comparison of Forecasting Models 6
3.4 Volume as an Additional Predictor 7
3.5 Daily Realised Volatility and Closing Price 7
PART 4- DATA VISUALIZATION **8**
4.1 PLOT 1- PRICE EVOLUTION (EXCHANGE 3) 8
4.2 PLOT 2 - DAILY VOLUME BY EXCHANGE 9
4.3 PLOT 3 - HOURLY REALIZED VARIANCE 9
4.4 PLOT 4- DAILY REALISED VOLATILITY 10
4.5 PLOT 5 - RV VS GARMAN-KLASS (COMPARISON) 10
APPENDIX **11**

PART 1 - DATA LOADING, CLEANING AND PLOTTING

1.1 Data Description

The dataset consists of high-frequency Bitcoin trade data for December 2018. Each observation includes a timestamp, transaction price, traded volume, and exchange identifier. The sample spans 1 December 2018 to 31 December 2018 and contains over 5 million trades across four exchanges.

The timestamp variable was cleaned to correct formatting inconsistencies and converted into a proper datetime index to enable time-based aggregation and resampling.

1.2 Price by Exchange

Objective:
To assess cross-exchange price consistency and identify potential anomalies or market dislocations.

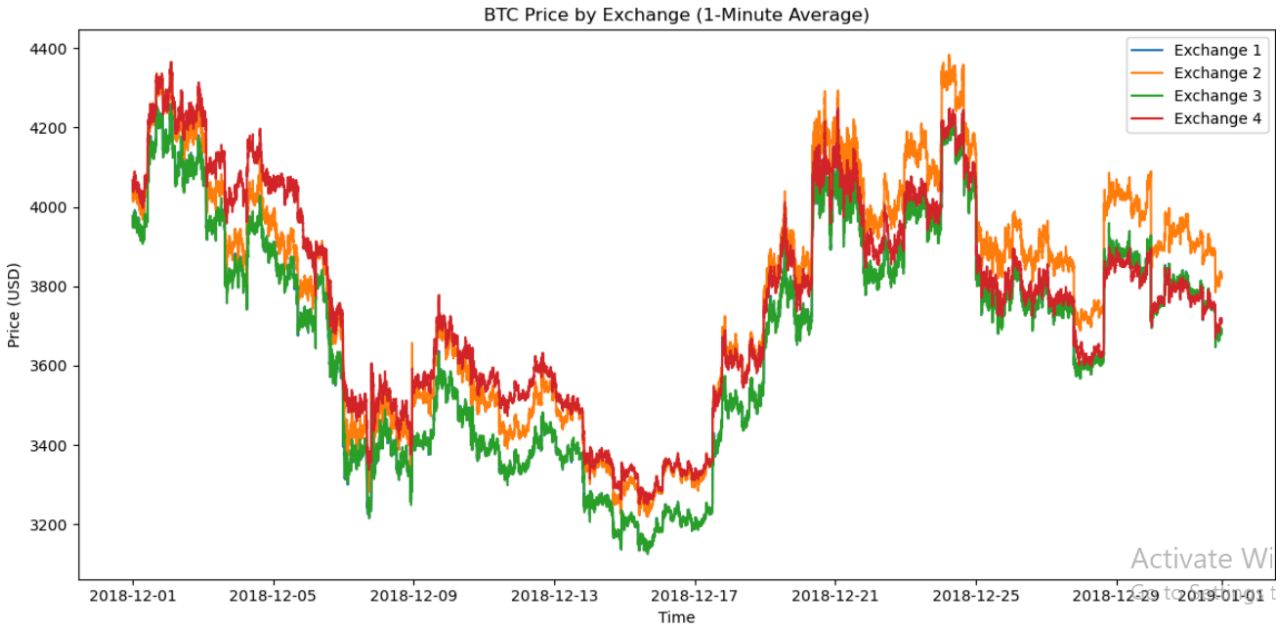


Figure 1: Hourly Average Bitcoin Price by Exchange (December 2018))

Short Analysis:

Prices across exchanges move closely together throughout the sample period, indicating strong market integration and efficient cross-exchange arbitrage. Minor temporary deviations are visible, likely reflecting short-lived liquidity differences.

1.3 Data Anomalies

Objective:
To visually and statistically detect irregular observations.

Extreme 1-minute returns were identified using a robust median absolute deviation filter. These events appear exchange-specific and likely reflect temporary liquidity shocks or microstructure noise. No non-positive prices or volumes were detected.

1.4 Daily Volume by Exchange

Objective:
To identify the most liquid exchange during the sample period.

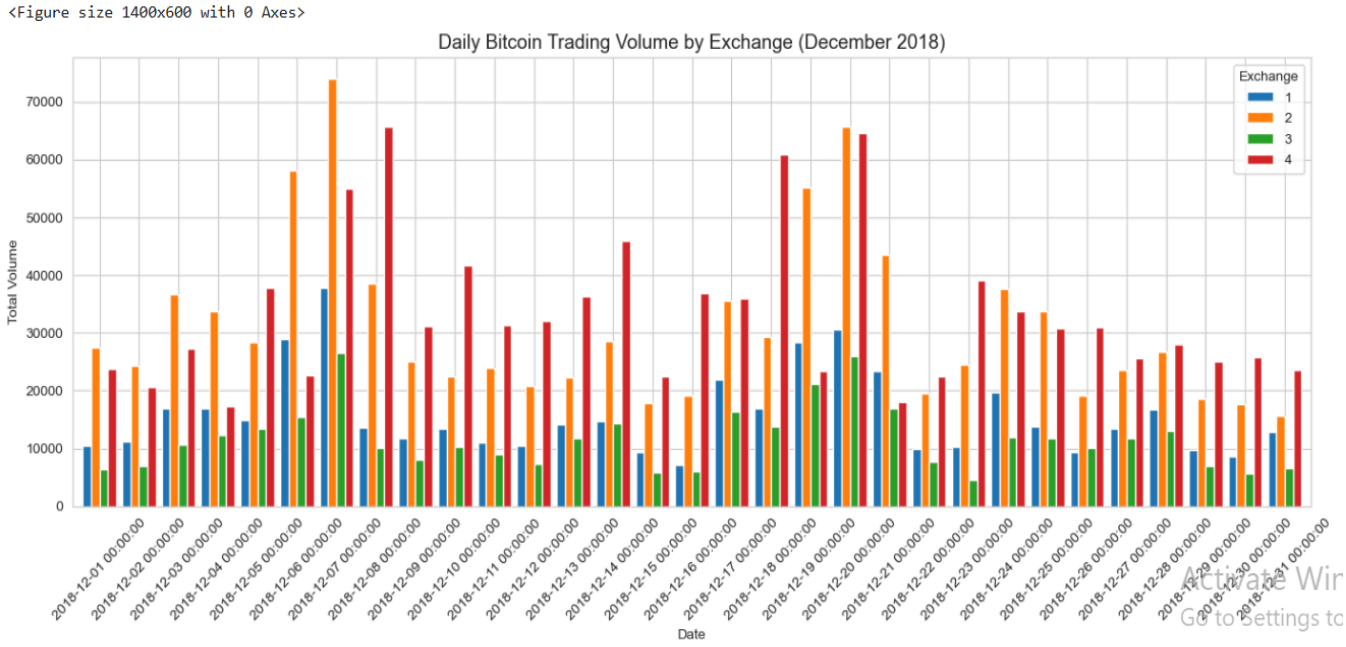


Figure 2: Daily Bitcoin Trading Volume by Exchange (December 2018))

Short Analysis:

Exchange 4 recorded the highest total trading volume during December 2018 and is therefore the most popular exchange by volume.

PART 2: 5-MINUTE VWAP & REALIZED VARIANCE

2.1 Five-Minute Price Construction

Five-minute volume-weighted prices were constructed using trade-level data aggregated into five-minute intervals across a continuous 24-hour clock. A total of **35,686 five-minute observations** were generated across four exchanges.

2.2 Missing Interval Analysis

Missing five-minute slots were counted for each exchange.

Exchange	Missing 5-min Intervals
1	21
2	1
3	0
4	4

Selection Decision:
Exchange 3 exhibited zero missing intervals and was selected for realised variance estimation to avoid interpolation bias.

Although no missing values were present, linear interpolation was formally applied to ensure methodological consistency.

2.3 Five-Minute Log Returns

Five-minute log returns were computed from the constructed five-minute prices. These returns capture short-horizon price fluctuations and serve as the basis for volatility estimation.

Number of 5-min observations: 8928

2.4 Hourly Realised Variance

Hourly realised variance was constructed by aggregating squared five-minute returns within each hour. This produced **744 hourly observations**, corresponding to 31 days × 24 hours.

Hourly realised variance provides a non-parametric measure of intraday volatility based solely on high-frequency price movements.

2.5 Daily Realised Volatility

Daily realised volatility was obtained by aggregating hourly realised variance at the daily level. A total of **31 daily volatility observations** were generated for December 2018.

The highest daily realised volatility occurred on:

7 December 2018- 6.99% - This indicates a significant increase in intraday price fluctuations on that date.

Related news:
Yes - “Dow tumbles more than 500 points, wipes out gain for year to cap wild week on Wall Street.” This broader market stress likely contributed to elevated Bitcoin volatility.

PART 3 - FORECASTING

3.1 Forecasting Hourly Realised Variance Using Lagged RV

A linear regression model was estimated to forecast the next-hour realised variance using lagged hourly realised variance.

The slope coefficient is positive (β = 0.121), indicating volatility persistence consistent with volatility clustering in financial markets. However, the model exhibits very weak predictive performance, with a near-zero and slightly negative out-of-sample R^2 (−0.010). This suggests that while volatility persistence exists, hourly realised variance remains largely unpredictable at this frequency.

3.2 Garman-Klass Volatility as Predictor

Hourly Garman-Klass volatility was constructed from OHLC price data and used as an alternative predictor of next-hour realised variance.

The Garman-Klass coefficient is positive (β = 0.306), indicating some persistence.
Out-of-sample performance marginally improves relative to lagged realised variance, with slightly lower MSE and a less negative R^2 (−0.008). However, predictive power remains weak overall

3.3 Comparison of Forecasting Models

Two forecasting models were estimated for next-hour realised variance using lagged realised variance and lagged Garman–Klass volatility. Both models exhibit limited predictive power, with near-zero and slightly negative out-of-sample R^2 values. The Garman–Klass estimator marginally outperforms lagged realised variance in terms of MSE, suggesting that range-based volatility measures contain slightly more information for short-horizon forecasting. However, hourly volatility remains largely unpredictable at this frequency.

```
Model A (Lagged RV)
Beta: 0.12061608414424306
MSE: 2.8604936081699855e-08
R^2: -0.010132755302977303

Model B (Lagged GK)
Beta: 0.3059416281485502
MSE: 2.8550916435466773e-08
R^2: -0.008225146982019593
```

Both forecasting models display limited explanatory power. Although the Garman-Klass estimator marginally outperforms lagged realised variance, the difference is economically small. These findings indicate that short-horizon volatility forecasting remains difficult in high-frequency cryptocurrency markets.

3.4 Volume as an Additional Predictor

A multivariate regression including lagged realised variance and lagged hourly volume was estimated.

Lagged hourly volume is statistically significant at the 5% level (p = 0.024), suggesting that trading activity contains incremental information about future volatility. In contrast, lagged realised variance becomes statistically insignificant once volume is included.

Despite statistical significance, overall explanatory power remains low (R^2 ~2%), and out-of-sample R^2 remains negative (−0.0015). This indicates that hourly volatility remains difficult to forecast even when incorporating market activity.

OLS Regression Results

Dep. Variable:	rv_t_plus_1	R-squared:	0.023
Model:	OLS	Adj. R-squared:	0.020
Method:	Least Squares	F-statistic:	6.966
Date:	Sat, 14 Feb 2026	Prob (F-statistic):	0.00102
Time:	13:45:41	Log-Likelihood:	4399.2
No. Observations:	594	AIC:	-8792.
Df Residuals:	591	BIC:	-8779.
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	5.952e-05	9.29e-06	6.405	0.000	4.13e-05	7.78e-05
rv_lag	0.0327	0.056	0.581	0.561	-0.078	0.143
vol_lag	4.186e-08	1.85e-08	2.265	0.024	5.57e-09	7.81e-08

Omnibus:	787.020	Durbin-Watson:	2.004
Prob(Omnibus):	0.000	Jarque-Bera (JB):	110374.029
Skew:	6.799	Prob(JB):	0.00
Kurtosis:	68.381	Cond. No.	6.34e+06

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 6.34e+06. This might indicate that there are strong multicollinearity or other numerical problems.

3.5 Daily Realised Volatility and Closing Price

A regression of daily closing prices on lagged daily realised volatility was estimated to assess whether volatility predicts future price levels.

```
                            OLS Regression Results
==============================================================================
Dep. Variable:            close_daily   R-squared:                       0.069
Model:                            OLS   Adj. R-squared:                  0.036
Method:                 Least Squares   F-statistic:                     2.068
Date:                Sat, 14 Feb 2026   Prob (F-statistic):              0.161
Time:                        13:49:00   Log-Likelihood:                -209.42
No. Observations:                  30   AIC:                             422.8
Df Residuals:                      28   BIC:                             425.7
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const       3375.3575    193.078     17.482      0.000    2979.854    3770.861
rvol_lag    6284.6519   4370.001      1.438      0.161   -2666.889    1.52e+04
==============================================================================
Omnibus:                        2.351   Durbin-Watson:                   0.493
Prob(Omnibus):                  0.309   Jarque-Bera (JB):                1.241
Skew:                          -0.070   Prob(JB):                        0.538
Kurtosis:                       2.013   Cond. No.                         89.0
==============================================================================

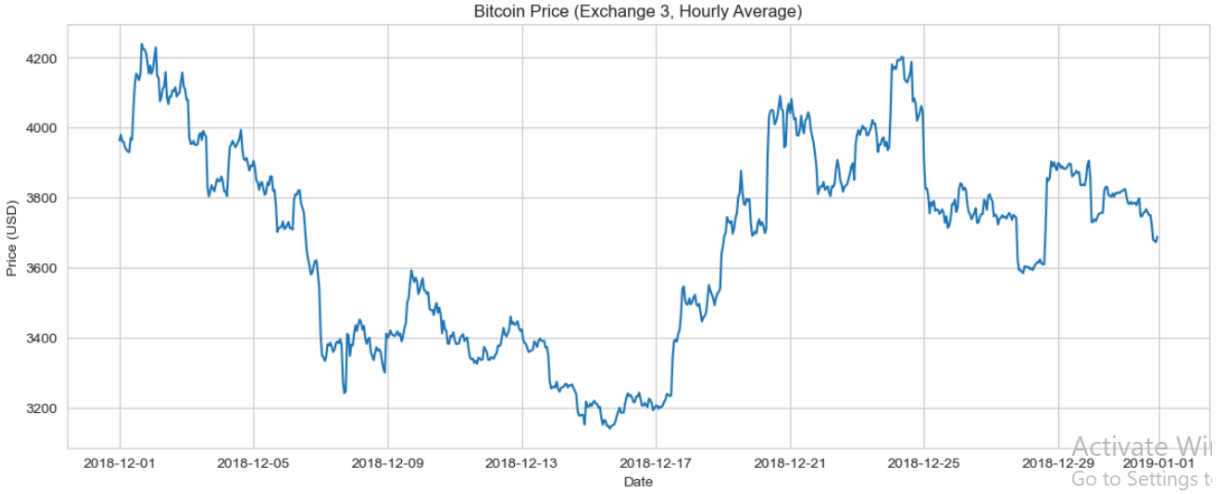
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Lagged daily realised volatility is not statistically significant (p = 0.161). The model explains approximately 7% of the variation in daily closing prices, and out-of-sample performance is poor ($R^2 = -1.75$).

This suggests that while volatility captures the magnitude of price fluctuations, it does not provide predictive information about future price direction. This finding is consistent with financial theory, where volatility reflects uncertainty rather than directional momentum.

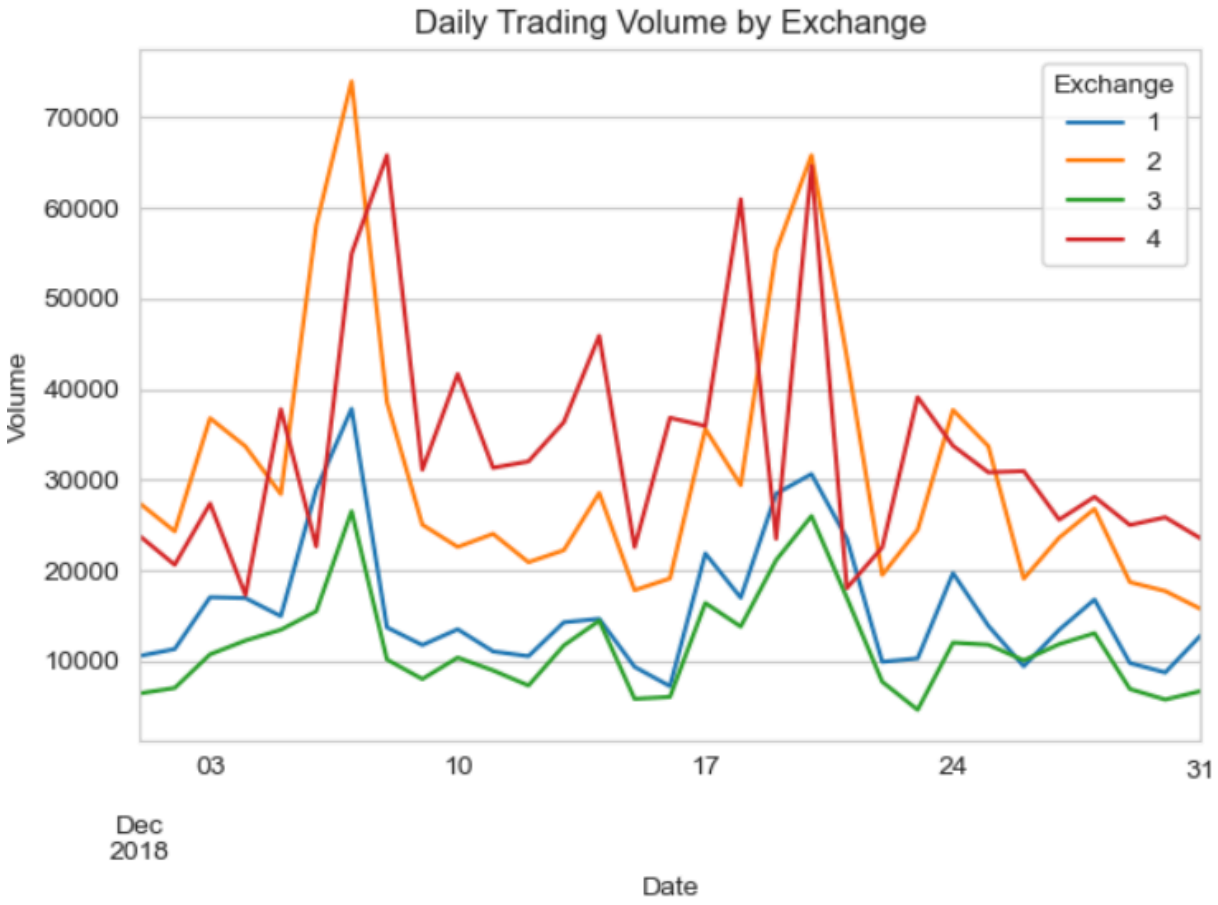
PART 4- DATA VISUALIZATION

4.1 PLOT 1- PRICE EVOLUTION (EXCHANGE 3)



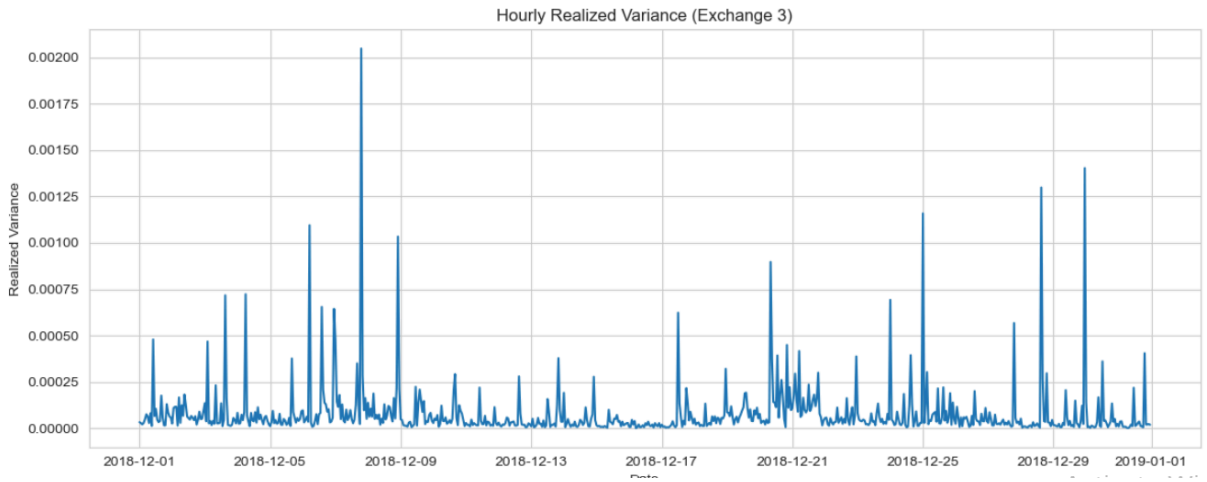
Bitcoin prices exhibit substantial intramonth variation, with a sharp decline in early December followed by a recovery phase mid-month. The pronounced swings reflect heightened market volatility consistent with broader financial market stress during the period.

4.2 PLOT 2 - DAILY VOLUME BY EXCHANGE



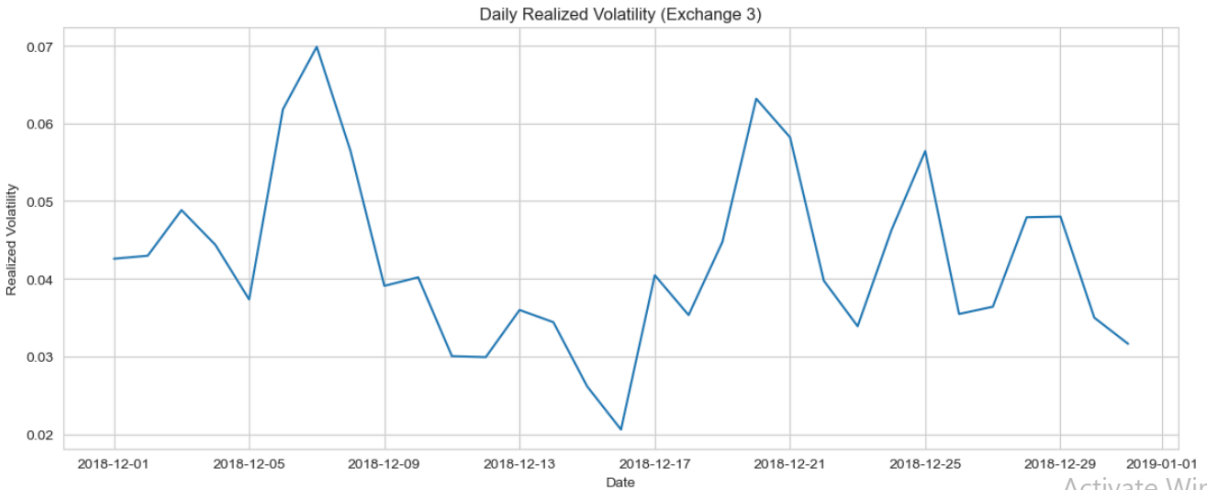
Trading volume varies substantially across exchanges, with Exchanges 4 and 2 consistently exhibiting higher activity levels. Volume spikes coincide with periods of elevated price volatility, suggesting intensified trading during market stress.

4.3 PLOT 3 - HOURLY REALIZED VARIANCE



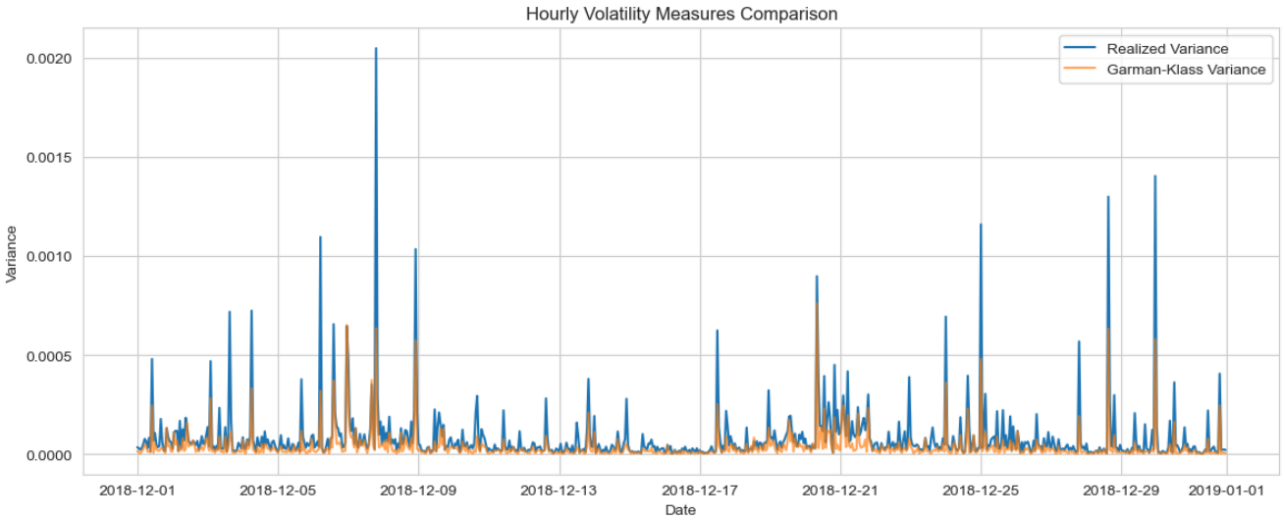
Hourly realised variance exhibits clear volatility clustering, with pronounced spikes in early and late December. These episodes reflect periods of intensified intraday price fluctuations consistent with market stress dynamics.

4.4 PLOT 4- DAILY REALISED VOLATILITY



Daily realised volatility peaks on 7 December 2018, indicating heightened market uncertainty during that period. Volatility remains elevated in mid-to-late December, reflecting persistent market instability.

4.5 PLOT 5 - RV VS GARMAN-KLASS (COMPARISON)



Both volatility measures exhibit similar clustering patterns, with spikes occurring during the same high-stress periods. The Garman–Klass estimator appears smoother and slightly less extreme, reflecting its range-based construction

The visual evidence confirms strong volatility clustering, liquidity concentration, and heightened market instability during early and late December 2018.

APPENDIX

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1. LOAD FILE (no header)
df = pd.read_csv("BTC_2018-12.txt", header=None)

# 2. ADD COLUMN NAMES
df.columns = ['timestamp', 'price', 'volume', 'exchange']
# 3. FIX TIMESTAMP FORMAT
# Change last ":" before milliseconds into "."
df['timestamp'] = df['timestamp'].astype(str).str.replace(
    r':(\d{3})$', r'.\1', regex=True
)
# 4. CONVERT TO DATETIME
df['timestamp'] = pd.to_datetime(
    df['timestamp'],
    format="%Y-%m-%d %H:%M:%S.%f"
)
# 5. SORT + SET INDEX
df = df.sort_values('timestamp')
df = df.set_index('timestamp')

# 6. CHECK
print(df.head())
print(df.info())


# Downsample to 1-minute average price for clean plotting
df_minute = df.groupby('exchange').resample('1min')['price'].mean().reset_index()
# Plotting
plt.figure(figsize=(12,6))

for ex in df_minute['exchange'].unique():
    subset = df_minute[df_minute['exchange'] == ex]
    plt.plot(subset['timestamp'], subset['price'], label=f'Exchange {ex}')

plt.title("BTC Price by Exchange (1-Minute Average)")
plt.xlabel("Time")
```

```
plt.ylabel("Price (USD)")
plt.legend()
plt.tight_layout()
plt.show()


#1. Basic data-quality checks
summary = {}

summary["rows"] = len(df)
summary["start"] = df.index.min()
summary["end"] = df.index.max()
summary["exchanges"] = sorted(df["exchange"].unique().tolist())
# duplicates in timestamp+exchange (trade-level duplicates)
summary["duplicate_timestamp_exchange_rows"] =
int(df.reset_index().duplicated(subset=["timestamp", "exchange"]).sum())
# non-positive / missing checks
summary["non_positive_price"] = int((df["price"] <= 0).sum())
summary["non_positive_volume"] = int((df["volume"] <= 0).sum())
summary["missing_price"] = int(df["price"].isna().sum())
summary["missing_volume"] = int(df["volume"].isna().sum())

# 1-minute VWAP per exchange
df1 = df.copy()

pv = (df1["price"] * df1["volume"]).groupby(df1["exchange"]).resample("1min").sum()
vv = (df1["volume"]).groupby(df1["exchange"]).resample("1min").sum()

vwap_1m = (pv / vv).rename("vwap").reset_index()
vwap_1m = vwap_1m.dropna(subset=["vwap"])
# 1-minute log returns per exchange (aligned)
vwap_1m = vwap_1m.sort_values(["exchange", "timestamp"])
vwap_1m["ret_1m"] = vwap_1m.groupby("exchange")["vwap"].transform(lambda s:
np.log(s).diff())

# Extreme-return flag using MAD z-score
def mad_zscore(x):
    x = np.asarray(x, dtype=float)
    med = np.nanmedian(x)
    mad = np.nanmedian(np.abs(x - med))
    if mad == 0 or np.isnan(mad):
        return np.zeros_like(x)
    return 0.6745 * (x - med) / mad

vwap_1m["z_mad"] = vwap_1m.groupby("exchange")["ret_1m"].transform(mad_zscore)

extreme = vwap_1m[np.abs(vwap_1m["z_mad"]) > 8].copy()
print("Extreme 1-min return events (|z_mad| > 8):", len(extreme))
#Cross-exchange divergence (max spread each minute)
px_pivot = vwap_1m.pivot(index="timestamp", columns="exchange", values="vwap")
spread = px_pivot.max(axis=1) - px_pivot.min(axis=1)
```

```
print("Max cross-exchange spread (USD):", float(spread.max()))
print("Timestamp of max spread:", spread.idxmax())
# Optional: showing top 10 extreme events
extreme_top10 = extreme.sort_values("z_mad", key=np.abs, ascending=False).head(10)
display(extreme_top10)

import matplotlib.pyplot as plt
import seaborn as sns

# Aggregate daily volume per exchange
daily_vol = (
    df.groupby("exchange")
      .resample("1D")["volume"]
      .sum()
      .reset_index()
)

# Create pivot table
daily_vol_pivot = daily_vol.pivot(
    index="timestamp",
    columns="exchange",
    values="volume"
)

# Professional styling
plt.figure(figsize=(14,6))
sns.set_style("whitegrid")

daily_vol_pivot.plot(
    kind="bar",
    width=0.85,
    figsize=(14,6)
)

plt.title("Daily Bitcoin Trading Volume by Exchange (December 2018)", fontsize=14)
plt.xlabel("Date")
plt.ylabel("Total Volume")
plt.xticks(rotation=45)
plt.legend(title="Exchange")
plt.tight_layout()
plt.show()

# Identifying the most popular exchange
monthly_volume = df.groupby("exchange")["volume"].sum().sort_values(ascending=False)

print("Monthly Total Volume by Exchange:")
print(monthly_volume)

print("\nMost Popular Exchange:", monthly_volume.index[0])

print("Dataset shape:", df.shape)
print("Date range:", df.index.min(), "to", df.index.max())
```

```
print("Number of exchanges:", df['exchange'].nunique())


# Making a copy first
df_5m = df.copy()

# Compute price * volume
df_5m["pv"] = df_5m["price"] * df_5m["volume"]

# Resample into 5-minute windows per exchange
pv_5m = df_5m.groupby("exchange").resample("5min")["pv"].sum()
vol_5m = df_5m.groupby("exchange").resample("5min")["volume"].sum()

# Computing VWAP
vwap_5m = (pv_5m / vol_5m).rename("vwap_5m").reset_index()

# Dropping intervals with no trades (division by zero - NaN)
vwap_5m = vwap_5m.dropna(subset=["vwap_5m"])
# Sort cleanly
vwap_5m = vwap_5m.sort_values(["exchange", "timestamp"])

print(vwap_5m.head())
print("\nTotal 5-min observations:", len(vwap_5m))

# Create full 5-min time index for the month
full_index = pd.date_range(
    start=vwap_5m["timestamp"].min(),
    end=vwap_5m["timestamp"].max(),
    freq="5min"
)

missing_summary = {}

for ex in vwap_5m["exchange"].unique():

    ex_data = vwap_5m[vwap_5m["exchange"] == ex].set_index("timestamp")

    # Reindex to full clock
    ex_reindexed = ex_data.reindex(full_index)

    missing_count = ex_reindexed["vwap_5m"].isna().sum()

    missing_summary[ex] = missing_count

print("Missing 5-min intervals per exchange:")
print(missing_summary)

print("\nExchange with least missing slots:",
      min(missing_summary, key=missing_summary.get))

ex_selected = 3
```

```
vwap_ex = (
    vwap_5m[vwap_5m["exchange"] == ex_selected]
    .copy()
    .set_index("timestamp")
)

print("Selected exchange:", ex_selected)
print("Number of 5-min observations:", len(vwap_ex))

vwap_ex.head()

# Reindex to full 5-minute clock (safety check)
full_index = pd.date_range(
    start=vwap_ex.index.min(),
    end=vwap_ex.index.max(),
    freq="5min"
)

vwap_ex = vwap_ex.reindex(full_index)

# Interpolate missing values (if any)
vwap_ex["vwap_5m"] = vwap_ex["vwap_5m"].interpolate(method="linear")

# Confirming no missing values remain
print("Remaining missing values:", vwap_ex["vwap_5m"].isna().sum())

vwap_ex["log_return"] = np.log(vwap_ex["vwap_5m"]).diff()

# Drop first NaN return
vwap_ex = vwap_ex.dropna()

print(vwap_ex.head())

# Square returns
vwap_ex["ret_sq"] = vwap_ex["log_return"]**2

# Aggregate to hourly level
rv_hourly = (
    vwap_ex["ret_sq"]
    .resample("1H")
    .sum()
    .to_frame(name="rv_hourly")
)

print(rv_hourly.head())
print("\nTotal hourly observations:", len(rv_hourly))

# Daily realized variance = sum of hourly RV
rv_daily_var = rv_hourly["rv_hourly"].resample("1D").sum().to_frame(name="rv_daily_var")

# Daily realized volatility = sqrt(daily realized variance)
rv_daily_var["rvol_daily"] = np.sqrt(rv_daily_var["rv_daily_var"])
```

```
print(rv_daily_var.head())
print("\nTotal daily observations:", len(rv_daily_var))

max_day = rv_daily_var["rvol_daily"].idxmax()
max_val = rv_daily_var.loc[max_day, "rvol_daily"]

print("Day with highest realized volatility:", max_day.date())
print("Highest daily rvol:", float(max_val))


from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# rv_hourly already created in Part 2.6 with column "rv_hourly"
# Ensure it's sorted and has no missing
rvh = rv_hourly.copy().sort_index().dropna()

# Build forecasting dataset: predict next hour using only lagged info
data_31 = pd.DataFrame(index=rvh.index)
data_31["rv_t"] = rvh["rv_hourly"]
data_31["rv_t_plus_1"] = rvh["rv_hourly"].shift(-1)  # target (next hour)

# Drop last row (no target)
data_31 = data_31.dropna()

# Features (X) and target (y)
X = data_31[["rv_t"]].values
y = data_31["rv_t_plus_1"].values

# Time-based split (no shuffling)
split = int(len(data_31) * 0.8)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Fit linear regression
lr = LinearRegression()
lr.fit(X_train, y_train)

# Predict + evaluate
y_pred = lr.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Model: RV(t+1) ~ RV(t)")
print("Intercept:", lr.intercept_)
print("Slope (beta):", lr.coef_[0])
print("Test MSE:", mse)
print("Test R^2:", r2)

# Optional: quick sanity table (last 10 forecasts in test)
```

```
out = data_31.iloc[split:].copy()
out["pred_rv_t_plus_1"] = y_pred
display(out.tail(10))

# vwap_ex: 5-min VWAP series for selected exchange (index = 5-min timestamps)
# Ensure we have a clean series
px5 = vwap_ex[["vwap_5m"]].copy().sort_index()

# 1) Hourly OHLC from 5-min VWAP
ohlc = px5["vwap_5m"].resample("1H").ohlc()
ohlc.columns = ["open", "high", "low", "close"]

# Optional: hourly volume (sum of trade volume in that hour for selected exchange)
# Build hourly volume from original df (trade-level) for that exchange
ex_selected = 3
hourly_vol = (
    df[df["exchange"] == ex_selected]["volume"]
    .resample("1H")
    .sum()
    .rename("vol_hourly")
)

# 2) Garman–Klass variance per hour
# GK variance: 0.5*(ln(H/L))^2 - (2ln2 - 1)*(ln(C/O))^2
ln_hl = np.log(ohlc["high"] / ohlc["low"])
ln_co = np.log(ohlc["close"] / ohlc["open"])

gk_var = 0.5 * (ln_hl ** 2) - (2 * np.log(2) - 1) * (ln_co ** 2)
gk_var = gk_var.rename("gk_var")

# Guardrail: occasional negative due to microstructure/noise - clip at 0
gk_var = gk_var.clip(lower=0)

# 3) Combine into one hourly feature table
features_hourly = pd.concat(
    [
        rv_hourly["rv_hourly"],   # from Part 2.6
        gk_var,
        hourly_vol
    ],
    axis=1
).dropna()

print(features_hourly.head())
print("\nRows:", len(features_hourly))

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Build forecasting dataset
forecast_df = features_hourly.copy()
```

```
forecast_df["rv_t_plus_1"] = forecast_df["rv_hourly"].shift(-1)
forecast_df["rv_lag"] = forecast_df["rv_hourly"]
forecast_df["gk_lag"] = forecast_df["gk_var"]

forecast_df = forecast_df.dropna()

# Train-test split (time based)
split = int(len(forecast_df) * 0.8)

train = forecast_df.iloc[:split]
test = forecast_df.iloc[split:]

# Model A: Lagged RV
X_train_A = train[["rv_lag"]]
X_test_A = test[["rv_lag"]]
y_train = train["rv_t_plus_1"]
y_test = test["rv_t_plus_1"]

model_A = LinearRegression()
model_A.fit(X_train_A, y_train)
pred_A = model_A.predict(X_test_A)

mse_A = mean_squared_error(y_test, pred_A)
r2_A = r2_score(y_test, pred_A)

# Model B: Lagged GK
X_train_B = train[["gk_lag"]]
X_test_B = test[["gk_lag"]]

model_B = LinearRegression()
model_B.fit(X_train_B, y_train)
pred_B = model_B.predict(X_test_B)

mse_B = mean_squared_error(y_test, pred_B)
r2_B = r2_score(y_test, pred_B)

print("Model A (Lagged RV)")
print("Beta:", model_A.coef_[0])
print("MSE:", mse_A)
print("R^2:", r2_A)

print("\nModel B (Lagged GK)")
print("Beta:", model_B.coef_[0])
print("MSE:", mse_B)
print("R^2:", r2_B)

import statsmodels.api as sm
from sklearn.metrics import mean_squared_error, r2_score

# Build dataset from features_hourly (created in 3.2)
df_vol = features_hourly.copy()
```

```
df_vol["rv_t_plus_1"] = df_vol["rv_hourly"].shift(-1)
df_vol["rv_lag"] = df_vol["rv_hourly"]
df_vol["vol_lag"] = df_vol["vol_hourly"]

df_vol = df_vol.dropna()

# Time-based split (80/20)
split = int(len(df_vol) * 0.8)
train = df_vol.iloc[:split]
test  = df_vol.iloc[split:]

# OLS with RV lag + Volume
X_train = train[["rv_lag", "vol_lag"]]
X_train = sm.add_constant(X_train)
y_train = train["rv_t_plus_1"]

model = sm.OLS(y_train, X_train).fit()

print(model.summary())

# Out-of-sample evaluation
X_test = sm.add_constant(test[["rv_lag", "vol_lag"]])
y_test = test["rv_t_plus_1"]

pred = model.predict(X_test)

mse = mean_squared_error(y_test, pred)
r2  = r2_score(y_test, pred)

print("\nOut-of-sample MSE:", mse)
print("Out-of-sample R^2:", r2)

from sklearn.metrics import mean_squared_error, r2_score

# 1) Daily close from 5-min VWAP (Exchange 3)
daily_close = (
    vwap_ex["vwap_5m"]
    .resample("1D")
    .last()
    .rename("close_daily")
)
# 2) Daily realized volatility from Part 2.7
# rv_daily_var has columns: ["rv_daily_var", "rvol_daily"]
daily_rvol = rv_daily_var["rvol_daily"].copy()

# 3) Build regression dataset
df_daily = pd.concat([daily_close, daily_rvol], axis=1).dropna()
df_daily["rvol_lag"] = df_daily["rvol_daily"].shift(1)
df_daily = df_daily.dropna()

# 4) OLS regression: Close_t ~ rvol_{t-1}
X = sm.add_constant(df_daily[["rvol_lag"]])
```

```
y = df_daily["close_daily"]

model_daily = sm.OLS(y, X).fit()
print(model_daily.summary())

# 5) (Optional) Out-of-sample split (80/20)
split = int(len(df_daily) * 0.8)
train = df_daily.iloc[:split]
test  = df_daily.iloc[split:]

X_train = sm.add_constant(train[["rvol_lag"]])
y_train = train["close_daily"]
X_test  = sm.add_constant(test[["rvol_lag"]])
y_test  = test["close_daily"]

model_oos = sm.OLS(y_train, X_train).fit()
pred = model_oos.predict(X_test)

mse = mean_squared_error(y_test, pred)
r2  = r2_score(y_test, pred)

print("\nOut-of-sample MSE:", mse)
print("Out-of-sample R^2:", r2)


sns.set_style("whitegrid")

plt.figure(figsize=(12,5))
plt.plot(vwap_ex["vwap_5m"].resample("1H").mean())
plt.title("Bitcoin Price (Exchange 3, Hourly Average)")
plt.xlabel("Date")
plt.ylabel("Price (USD)")
plt.tight_layout()
plt.show()

plt.figure(figsize=(12,5))
daily_vol_pivot.plot()
plt.title("Daily Trading Volume by Exchange")
plt.xlabel("Date")
plt.ylabel("Volume")
plt.legend(title="Exchange")
plt.tight_layout()
plt.show()

plt.figure(figsize=(12,5))
plt.plot(rv_hourly["rv_hourly"])
plt.title("Hourly Realized Variance (Exchange 3)")
plt.xlabel("Date")
plt.ylabel("Realized Variance")
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(12,5))
plt.plot(rv_daily_var["rvol_daily"])
plt.title("Daily Realized Volatility (Exchange 3)")
plt.xlabel("Date")
plt.ylabel("Realized Volatility")
plt.tight_layout()
plt.show()

plt.figure(figsize=(12,5))
plt.plot(features_hourly["rv_hourly"], label="Realized Variance")
plt.plot(features_hourly["gk_var"], label="Garman-Klass Variance", alpha=0.7)
plt.title("Hourly Volatility Measures Comparison")
plt.xlabel("Date")
plt.ylabel("Variance")
plt.legend()
plt.tight_layout()
plt.show()
```